Яковець Арина, ФФ-41

ЛАБОРАТОРНА РОБОТА №6. Наближене обчислення інтегралів

Варіант 15

Мета: Закріплення знань із застосування методів чисельного інтегрування функцій.

---

15. а) Обчислити наближено інтеграл $\int_0^1 cos(x^2)dx$ за формулою Сімпсона при $n=10$.

Оцінити залишковий член;

Код реалізації:

```
import math

def f(x):
    return math.cos(x**2)
def f4(x):
    return (16*x**4 - 12)*math.cos(x**2) + 48*x**2*math.sin(x**2)

a = 0
b = 1
n = 10

h = (b - a) / n

x_values = []
y_values = []

for i in range(n + 1):
    x = a + i*h
    y = f(x)
    x_values.append(x)
    y_values.append(y)

sum_odd = 0
sum_even = 0

for i in range(1, n):
    if i % 2 == 1:
        sum_odd += y_values[i]
    else:
        sum_even += y_values[i]

I = (h / 3) * (y_values[0] + y_values[n] + 4*sum_odd + 2*sum_even)

# оцінка залишкового члена
points = 100000
M4 = 0

for k in range(points + 1):
    x = a + (b - a) * k / points
    value = abs(f4(x))
    if value > M4:
        M4 = value

R = ((b - a) * h**4 / 180) * M4

print("I =", I)
print("|R2| <=", R)
```

Результат:

```
I = 0.9045242678623499
|R2| <= 2.3639898052361996e-05
```

б) Обчислити інтеграл $\int_1^2 \frac{lg\, x}{x} dx$ за формулою трапецій з точністю до 10^{-2}. Величину кроку h, що забезпечує потрібну точність, визначити за допомогою подвійного перерахунку;

Код реалізації:

```
def f(x):
    return math.log10(x) / x
a = 1
b = 2
eps = 1e-2

def trapezoid(n):
    h = (b - a) / n
    y0 = f(a)
    yn = f(b)

    sum_middle = 0
    for i in range(1, n):
        xi = a + i*h
        sum_middle += f(xi)

    I = (h / 2) * (y0 + 2*sum_middle + yn)
    return I

# початкове n
n = 10

while True:
    I_n = trapezoid(n)
    I_2n = trapezoid(2*n)

    delta = abs(I_n - I_2n) / 3

    print(f"n ={n:3d}, I_n = {I_n:.6f}, I_2n = {I_2n:.6f}, delta ≈ {delta:.6f}")

    if delta < eps:
        print(f"h = {(b - a)/(2*n)}")
        print(f"I ≈ {I_2n}") #точне значення
        print(f"I округ ≈ {round(I_2n, 2)}") #округлене
        break

    n *= 2
```

Результат:

```
n = 10, I_n = 0.103996, I_2n = 0.104246, delta ≈ 0.000083
h = 0.05
I ≈ 0.10424554901737937
I округ ≈ 0.1
```

в) За допомогою розкладу підінтегральної функції у степеневий ряд обчислити з точністю до 10^{-4} інтеграл $\int_{0.5}^{0.8} \sqrt{1+x^3}\,dx$;

Код реалізації:

```
a = 0.5
b = 0.8
eps = 1e-4

terms = [
    (1, 0),
    (1/2, 3),
    (-1/8, 6),
    (1/16, 9),
    (-5/128, 12),
]

I = 0

for coef, power in terms:
    integral_term = coef * (b**(power + 1) - a**(power + 1)) / (power + 1)
    I += integral_term

print(f"I ≈ {I:.8f}")
print(f"I ≈ {I:.4f}")

# наступний відкинутий член
next_coef = 7/256
next_power = 15

R = abs(next_coef) * (b**(next_power + 1) - a**(next_power + 1)) / (next_power + 1)

print(f"|R| ≤ {R:.8e}")

if R < eps:
    print("Точність 10^-4 є")
else:
    print("Потрібно взяти більше членів ряду")
```

Результат:

```
I ≈ 0.34028226
I ≈ 0.3403
|R| ≤ 4.80775567e-05
Точність 10^-4 є
```

г) Обчислити інтеграл $\int_0^{\infty} \frac{\sin x}{x+1} e^{-x}\,dx$ для $n = 3,4,5$, використовуючи квадратурні формули з вагою;

Код реалізації:

```python
def phi(x):
    return math.sin(x) / (x + 1)

# таблиця Чебишева-Лагерра:
data = {
    3: [
        (0.415774, 0.711093),
        (2.294280, 0.278518),
        (6.289945, 0.010389),
    ],
    4: [
        (0.322548, 0.603154),
        (1.745761, 0.357419),
        (4.536620, 0.038888),
        (9.395071, 0.000539),
    ],
    5: [
        (0.263560, 0.521756),
        (1.413403, 0.398667),
        (3.596425, 0.075942),
        (7.085810, 0.003612),
        (12.640801, 0.000023),
    ]
}

for n in [3, 4, 5]:
    I = 0

    print(f"\nn = {n}")

    for xk, Ak in data[n]:
        term = Ak * phi(xk)
        I += term

    print(f"I_{n} ≈ {I:.6f}")
```

Результат:

```
n = 3
I_3 ≈ 0.266241

n = 4
I_4 ≈ 0.265832

n = 5
I_5 ≈ 0.263785
```

д) Обчислити інтеграл $\int_0^1 dx \int_1^2 \frac{\sin\left(x^2+y^2\right)}{1+0.9x+0.9y}\,dy$ методом повторного інтегрування, застосовуючи формулу Сімпсона при $n_x = n_y = 10$.

Код реалізації:

```
def f(x, y):
    return math.sin(x**2 + y**2) / (1 + 0.9*x + 0.9*y)
a = 0
b = 1
c = 1
d = 2
nx = 10
ny = 10
hx = (b - a) / nx
hy = (d - c) / ny

def simpson_y(x):
    y0 = f(x, c)
    yn = f(x, d)

    sum_odd = 0
    sum_even = 0

    for j in range(1, ny):
        y = c + j * hy

        if j % 2 == 1:
            sum_odd += f(x, y)
        else:
            sum_even += f(x, y)

    F = (hy / 3) * (y0 + yn + 4*sum_odd + 2*sum_even)
    return F

F0 = simpson_y(a)
Fn = simpson_y(b)

sum_odd = 0
sum_even = 0

for i in range(1, nx):
    x = a + i * hx
    Fi = simpson_y(x)

    if i % 2 == 1:
        sum_odd += Fi
    else:
        sum_even += Fi

I = (hx / 3) * (F0 + Fn + 4*sum_odd + 2*sum_even)

print(f"I ≈ {I:.6f}")
```

Результат:

```
==============
I ≈ 0.138060
```